

CENTINEL

Primera Auditoria Integral del Sistema

Sistema de Monitoreo Electoral -- Honduras

Fecha: 07 de junio de 2026

Clasificacion: Confidencial -- Uso interno

Equipo auditor: 12 agentes especializados

Seguridad | Criptografia | Infraestructura | Backend | UX | Compliance

Arquitectura | Red Team | Integridad de datos | DevOps

Contenido

1. Resumen ejecutivo
2. Hallazgos criticos y de alta severidad
3. Seguridad y criptografia
4. Integridad de datos electorales
5. Infraestructura y DevOps
- 6.Codigo Python -- Backend
7. Panel OPS -- Frontend
8. UX y accesibilidad
9. Compliance y documentacion
10. Resumen de hallazgos por severidad
11. Recomendaciones prioritarias
12. Conclusion y hoja de ruta

1. RESUMEN EJECUTIVO

Centinel es un sistema de monitoreo electoral independiente para Honduras, diseñado como sitio estatico servido via GitHub Pages sin backend. Toda la logica de operacion corre en el navegador del operador.

Esta auditoria evaluo 12 dimensiones del sistema usando agentes especializados que revisaron ~65,000 lineas de codigo (Python + JS + CSS + HTML), 23 workflows CI/CD, infraestructura Docker, documentacion, y compliance legal.

El sistema demuestra una arquitectura de seguridad multicapa impresionante para un proyecto de su escala: cadena de hashes SHA-256, firmas Ed25519, anclaje Bitcoin via OpenTimestamps, cifrado AES-256-GCM con key wrapping HKDF, y autenticacion PBKDF2 client-side. Sin embargo, se identificaron hallazgos que requieren atencion antes de una aplicacion formal a grants internacionales.

Metricas clave

- * Lineas de codigo: ~32,400 (Python src) + ~4,300 (JS ops) + ~536 (CSS)
- * Tests: 81 archivos, ~15,400 lineas (ratio 1:2 test:src)
- * Workflows CI/CD: 23 en `.github/workflows/`
- * Agentes de auditoria en el repo: 18 especializados
- * Dependencias Python: ~35 directas, pinned con bounds
- * Costo operativo: \$0 (GitHub Pages + Actions free tier)

2. HALLAZGOS CRITICOS Y DE ALTA SEVERIDAD

CRITICAL

Ruta v3 degrada autenticacion a SHA-256 simple

web/ops/index.html:1488-1491

Si access.json tiene version ≥ 3 o method === "SHA-256", la autenticacion usa SHA-256 directo (1 iteracion), eliminando completamente la proteccion de PBKDF2. Un atacante haria fuerza bruta sobre los ~25.6M seeds en segundos.

Impacto: Bypass completo de autenticacion en segundos con GPU commodity.

Rec: Eliminar la ruta v3/SHA-256. Mantener siempre PBKDF2 $\geq 600k$ iteraciones.

HIGH

Salt PBKDF2 estatico compartido entre instalaciones

web/setup/index.html:784

PBKDF2_SALT = "centinel-admin-salt-v1" es identico en toda instalacion. Permite rainbow tables pre-computadas reutilizables contra cualquier instalacion.

Impacto: Rainbow table de 25.6M hashes, una sola vez, reutilizable globalmente.

Rec: Generar salt aleatorio de 16 bytes por instalacion, almacenar en access.json.

HIGH

access.json publico + entropia baja = brute-force viable

web/access.json (publico en GitHub Pages)

access.json contiene hashes PBKDF2 y esta servido publicamente. Combinado con 24.6 bits de entropia por seed (40 adj x 64 nouns x 10000 digitos), un atacante con GPU (RTX 4090) recorre todo el espacio en ~85 segundos.

Impacto: Recuperacion de seeds y descifrado del PAT del dueño de la cuenta.

Rec: Ampliar wordlists a 2048 palabras (estandar BIP-39), elevando entropia a ~35+ bits.

HIGH

Inconsistencia en formatos de hash chain

src/centinel/core/hashchain.py vs src/centinel/download.py

Dos implementaciones con dominios diferentes (centinel-hashchain-v1 vs centinel-chain-v1) que producen hashes diferentes para los mismos datos. Un verificador externo podria reportar la cadena como rota.

Impacto: Perdida de credibilidad forense ante auditores externos.

Rec: Consolidar en una sola implementacion con dominio versionado unico.

HIGH

subprocess.Popen sin sanitizacion en watchdog

scripts/watchdog.py:458, scripts/watchdog_daemon.py:402

pipeline_command viene de configuracion YAML. Un atacante que modifique el archivo de configuracion podria ejecutar comandos arbitrarios.

Impacto: Ejecucion remota de codigo si se compromete el archivo de configuracion.

Rec: Validar pipeline_command contra allowlist de comandos permitidos.

HIGH

CI solo ejecuta 2 de 81 archivos de test

.github/workflows/ci.yml:92-94

El job principal de tests solo corre test_hashchain.py y test_turnout_impossible_rule.py. El 97% de los tests no corren en CI.

Impacto: Bugs y regresiones pasan a produccion sin detectarse.

Rec: Ejecutar la suite completa con pytest.

HIGH

Clave privada con formato real en .env.example

centinel_engine/.env.example:29 y .env.example:8

PRIVATE_KEY=0xTU_CLAVE_PRIVADA y GITHUB_TOKEN=ghp_... usan formatos que se parecen a valores reales.

Impacto: Filtracion accidental de credenciales.

Rec: Usar placeholders que no parezcan reales: <your-key-here>.

3. SEGURIDAD Y CRIPTOGRAFIA

3.1 Autenticacion (PBKDF2)

Parametros: PBKDF2-HMAC-SHA-256, 600,000 iteraciones. Conforme a NIST SP 800-132 y OWASP 2023.

Fuente de entropia: `crypto.getRandomValues()` (Web Crypto CSPRNG). Correcto.

Debilidades: salt estatico global, entropia de seed baja (~24.6 bits), ruta v3 SHA-256 directa.

3.2 Cifrado PAT (AES-256-GCM + HKDF)

Arquitectura correcta: master key AES-256-GCM aleatoria cifra el PAT, luego se wrappea una vez por seed via HKDF(PBKDF2_hash). IV de 12 bytes (96 bits) por operacion GCM.

Conforme a NIST SP 800-38D. HKDF con domain separation conforme a RFC 5869.

Sin hallazgos de severidad en esta capa.

3.3 Firmas Ed25519

Sistema completo: firma por operador en cada snapshot. Clave privada protegida con passphrase AES-256-CBC, permisos 0o600.

Verificacion al arranque del pipeline.

3.4 Anclaje temporal

Bitcoin via OpenTimestamps + Arbitrum via smart contract. Diseno de defensa en profundidad.

Hallazgo MEDIUM: la verificacion de prueba OTS es superficial -- no valida la estructura binaria.

MEDIUM

Verificacion OTS superficial

`src/centinel/anchor/opentimestamps_client.py:223-253`

`verify_proof()` solo verifica que `ots_response` no este vacio. No decodifica la estructura binaria OTS.

Impacto: Un servidor de calendario malicioso podria devolver datos arbitrarios.

Rec: Integrar python-opentimestamps o validar header magico OTS.

MEDIUM

Dos implementaciones OTS con APIs incompatibles

`src/anchor/opentimestamps.py` vs `src/centinel/anchor/opentimestamps_client.py`

Una usa `requests+POST` binario, la otra `httpx+POST` JSON. Solo el formato binario es estandar.

Impacto: La implementacion JSON podria fallar contra calendarios OTS reales.

Rec: Consolidar en una sola implementacion usando protocolo binario estandar.

4. INTEGRIDAD DE DATOS ELECTORALES

4.1 Hash chain

SHA-256 con encadenamiento estructurado. Incluye prefijo de dominio, separadores de longitud, y hash anterior. Previene ataques de extension de longitud.

Problema: dos implementaciones con dominios diferentes (ver hallazgo HIGH en seccion 2).

MEDIUM

Bundle de evidencia sin firma ni timestamp

`scripts/evidence_bundle.py:46-56`

El bundle generado no incluye firma Ed25519 ni prueba OTS.

Impacto: Un atacante con acceso de escritura podría regenerar un bundle falso.

Rec: Firmar el bundle con clave Ed25519 y anclar Merkle root via OTS.

4.2 Deteccion de anomalias

Benford: chi-cuadrado con 8 GL, umbral 5.99 ($p < 0.05$). Minimo 20 primeros digitos. Correcto.

Z-score de 3.0 sigma para deltas de votos: estandar estadistico apropiado.

Monotonicidad de timestamps: detecta manipulacion temporal.

LOW

Umbral Benford fijo sin ajuste por tamano de muestra

`src/centinel/core/anomaly_detector.py:96`

`benford_chi2_threshold=5.99` es fijo. Para muestras muy grandes ($>10,000$), desviaciones insignificantes generarian alertas.

Impacto: Falsos positivos en jornadas electorales con muchos datos.

Rec: Agregar test de efecto (Cramer V) ademas del chi-cuadrado.

5. INFRAESTRUCTURA Y DEVOPS

5.1 CI/CD

23 workflows en .github/workflows/. Pipeline principal incluye lint, tests (matrix Python 3.10/3.11), tests de seguridad con SBOM (CycloneDX), release gate, y resiliencia.

Permissions explicitas y minimalistas. timeout-minutes definido. concurrency con cancel-in-progress.

MEDIUM

Workflows secundarios sin SHA pinning (~12)

build-launchers.yml, gpu-batch-validation.yml, codeql.yml, etc.

Usan tags flotantes (@v4, @v5). Riesgo de supply-chain attack.

Impacto: Código malicioso ejecutado en el runner si una Action es comprometida.

Rec: Pinear todas las Actions a SHA de commit.

MEDIUM

Bandit y security gates son soft-fail en PRs

.github/workflows/ci.yml:178-191

bandit y make security usan || true en PRs y ramas dev.

Impacto: Vulnerabilidades conocidas pueden pasar a produccion via dev.

Rec: Hacer security gates obligatorios al menos en PRs a main.

5.2 Docker

Multi-stage build, usuario no-root, HEALTHCHECK configurado. docker-compose.yml con healthchecks, limites de recursos (CPU 1.0, RAM 1G), red interna, rotacion de logs.

MEDIUM

Docker socket montado en watchdog

docker-compose.yml:120

El watchdog monta /var/run/docker.sock:ro. Permite enumeracion de contenedores.

Impacto: Posible escape de contenedor en entornos compartidos.

Rec: Evaluar alternativas al socket mount para el watchdog.

6. CODIGO PYTHON -- BACKEND

6.1 Seguridad del codigo

Buenas practicas: `yaml.safe_load()` consistente, no hay `pickle/eval/exec`, SQL injection mitigada con `_assert_safe_identifier()`, headers sensibles redactados, TLS context estricto, validacion anti-SSRF.

MEDIUM

Excepciones genericas silenciadas con pass

`custody.py:405, swarm.py:466, pipeline.py:87-88`

except Exception: pass oculta errores criticos en produccion.

Impacto: Fallos silenciosos en operaciones de integridad electoral.

Rec: Capturar excepciones especificas o al menos loguear el error.

MEDIUM

Inconsistencia de dependencias

`requirements.txt` vs `pyproject.toml` vs `requirements-prod.txt`

Versiones diferentes de aiohttp, dependencias faltantes entre archivos.

Impacto: Builds no reproducibles entre entornos.

Rec: Definir una fuente canonica (`pyproject.toml`) y generar los demas desde ella.

6.2 Arquitectura

Separacion clara: defense, core, api, federation. Reglas como modulos pluggables con registry pattern.

Problemas: hash chain implementada en 3 ubicaciones con formatos diferentes, 4 mecanismos de configuracion sin punto unico de verdad.

7. PANEL OPS -- FRONTEND

7.1 Arquitectura

Estructura modular: index.html (shell + globals + auth) + ops.css + 3 JS files.

Load order: ops-core.js -> ops-panel.js -> ops-monitor.js -> inline script.

Global scope via script src -- no ES modules. ~152 funciones, onclick handlers en HTML.

SpaceX design system consistente: CSS variables, 2px radius, uppercase labels, monospace data.

MEDIUM

innerHTML con datos de API sin sanitizacion completa

web/ops/js/ops-panel.js, ops-monitor.js (multiples ubicaciones)

Template literals insertados via innerHTML. La funcion esc() cubre &, <, > pero no atributos ni comillas. Datos de GitHub API (commit messages) podrian contener payloads XSS.

Impacto: XSS almacenado si un commit message contiene JavaScript malicioso.

Rec: Usar textContent donde sea posible. Extender esc() para cubrir comillas.

LOW

Sin Content Security Policy

web/ops/index.html (GitHub Pages no permite headers custom)

No hay meta tag CSP. Scripts inline y eval() no estan restringidos.

Impacto: XSS tendria acceso completo al DOM y sessionStorage (PAT).

Rec: Agregar meta http-equiv="Content-Security-Policy" con script-src restrictivo.

8. UX Y ACCESIBILIDAD

8.1 Accesibilidad (WCAG 2.1 AA)

Contraste: --fg (#e7e9ec) sobre --bg (#080b0f) = ratio ~16:1 (excelente).

--muted (#8b929c) sobre --bg = ratio ~5.5:1 (pasa AA para texto normal).

--warn (#d4b066) sobre --bg = ratio ~7:1 (bueno).

Problema: --accent (#6ea8fe) sobre --panel (#0d1117) = ~4.2:1 (falla AA para texto pequeño).

MEDIUM

Sin ARIA labels en controles interactivos

web/ops/index.html (sliders, toggles, tabs)

Los sliders no tienen aria-label ni aria-valuetext. Los tabs no usan role="tablist"/role="tab".

Impacto: Inutilizable para usuarios con lectores de pantalla.

Rec: Agregar roles ARIA y labels descriptivos.

MEDIUM

Sin manejo de focus en modales

web/ops/js/ops-panel.js -- showDiffModal()

Al abrir el diff modal, el focus no se mueve al modal. No hay focus trap.

Impacto: Usuarios de teclado no pueden navegar el modal eficientemente.

Rec: Implementar focus trap y mover focus al modal al abrirlo.

LOW

Touch targets menores a 44x44px

web/ops/ops.css -- botones de header, tabs

Varios botones tienen padding insuficiente para uso táctil/móvil.

Impacto: Dificultad de uso en dispositivos móviles.

Rec: Aumentar padding a mínimo 44x44px en elementos táctiles.

9. COMPLIANCE Y DOCUMENTACION

9.1 Documentacion existente

- * README.md: existe, explica el proyecto, setup, uso basico.
- * CLAUDE.md: documenta arquitectura del panel ops, design system, convenciones.
- * docs/dev-diary/: diario de desarrollo con convenciones claras.
- * docs/grants/: BUDGET_NARRATIVE.md presente.
- * docs/architecture/: THEORY_OF_CHANGE.md presente.
- * SECURITY.md: presente con politica de reporte de vulnerabilidades.

MEDIUM

Sin manual de operador documentado

docs/ (ausente)

No hay guia paso a paso para operadores electorales. El panel es intuitivo pero los procedimientos de cadena de custodia, reportes, y emergencias no estan documentados.

Impacto: Operadores sin capacitacion podrian cometer errores en jornada electoral.

Rec: Crear docs/OPERATOR_MANUAL.md con procedimientos para cada escenario.

MEDIUM

heal_web.py no incluye todos los archivos criticos

scripts/heal_web.py -- CRITICAL_FILES

Verificar que ops-core.js, ops-panel.js, ops-monitor.js, ops.css estan en la lista.

Impacto: Archivos criticos podrian no restaurarse ante corrupcion.

Rec: Auditar y actualizar CRITICAL_FILES.

9.2 Grant readiness

Fortalezas:

- * Costo operativo \$0 (GitHub free tier) -- atractivo para donantes
- * Arquitectura de seguridad multicapa documentable
- * Theory of Change y Budget Narrative presentes
- *Codigo abierto con CI/CD, SBOM, auditoria de secretos
- * 18 agentes especializados demuestran capacidad tecnica

Brechas:

- * Sin M&E framework con indicadores SMART
- * Sin manual de operador
- * Hallazgos de seguridad pendientes (CRITICAL + HIGH)
- * Sin evidencia de piloto o deployment real
- * Sin letters of support de organizaciones locales
- * Sin plan de sostenibilidad post-grant documentado

10. RESUMEN DE HALLAZGOS POR SEVERIDAD

Severidad	Cant.	Hallazgos clave
CRITICAL	1	Ruta v3 degrada auth a SHA-256 simple (1 iteracion)
HIGH	6	Salt estatico, entropia baja, hash chain, CI, .env
MEDIUM	14	OTS superficial, bundles sin firma, ARIA, XSS parcial
LOW	10	CSP ausente, touch targets, skip-link, umbral Benford
INFO	15	Buenas practicas: Ed25519, AES-GCM, SBOM, healthchecks

Total: 46 hallazgos documentados.

1 CRITICAL + 6 HIGH requieren atencion inmediata.

14 MEDIUM deberian resolverse antes de aplicar a grants.

10 LOW son mejoras deseables pero no bloqueantes.

11. RECOMENDACIONES PRIORITARIAS

P0 -- Inmediato (1-2 semanas)

- * Eliminar la ruta v3/SHA-256 directa en web/ops/index.html:1488-1491
- * Generar salt aleatorio por instalacion en web/setup/index.html
- * Ampliar wordlists de seeds a 2048 palabras (BIP-39)
- * Consolidar hash chain en una sola implementacion
- * Expandir suite de tests en CI (ejecutar todos los tests)

P1 -- Antes de grant (1 mes)

- * Crear manual de operador (docs/OPERATOR_MANUAL.md)
- * Implementar validacion completa de pruebas OTS
- * Firmar bundles de evidencia con Ed25519
- * Agregar ARIA labels y focus management en modales
- * Pinear todas las GitHub Actions a SHA
- * Crear .dockerignore
- * Hacer security gates obligatorios en PRs a main

P2 -- Mejora continua

- * Agregar Content Security Policy via meta tag
- * Extender sanitizacion XSS (cubrir atributos y comillas)
- * Agregar tests para rutas API criticas
- * Documentar M&E framework con indicadores SMART
- * Estandarizar typing a estilo moderno (Python 3.10+)
- * Evaluar Docker secrets vs env vars para secretos

12. CONCLUSION Y HOJA DE RUTA

Centinel es un proyecto con una base tecnica solida y ambiciosa. La arquitectura de seguridad multicapa (PBKDF2 + AES-256-GCM + HKDF + Ed25519 + OTS + Arbitrum) demuestra un enfoque serio hacia la integridad de datos electorales.

Los hallazgos CRITICAL y HIGH son corregibles en un sprint de 1-2 semanas. Los MEDIUM requieren ~1 mes de trabajo. Una vez resueltos los P0 y P1, el sistema estaria en posicion competitiva para aplicar a grants de organizaciones como NED, OTF, o fondos EU.

La mayor fortaleza del proyecto es su modelo de costo cero: al operar enteramente sobre GitHub Pages y Actions free tier, elimina la barrera de sostenibilidad financiera que preocupa a los donantes.

La mayor debilidad actual es la falta de un piloto real con datos electorales. Un piloto exitoso (incluso con datos historicos del TSE) transformaria la propuesta de "prometedora" a "probada", aumentando dramaticamente las posibilidades de financiamiento.

Recomendacion final: resolver los hallazgos P0 inmediatamente, luego ejecutar un piloto con datos de las elecciones generales de Honduras 2025, documentar resultados, y aplicar a grants con evidencia concreta de funcionamiento.

-- Fin del reporte --

Generado: 2026-06-07 12:32 UTC

Equipo: 12 agentes especializados + coordinador